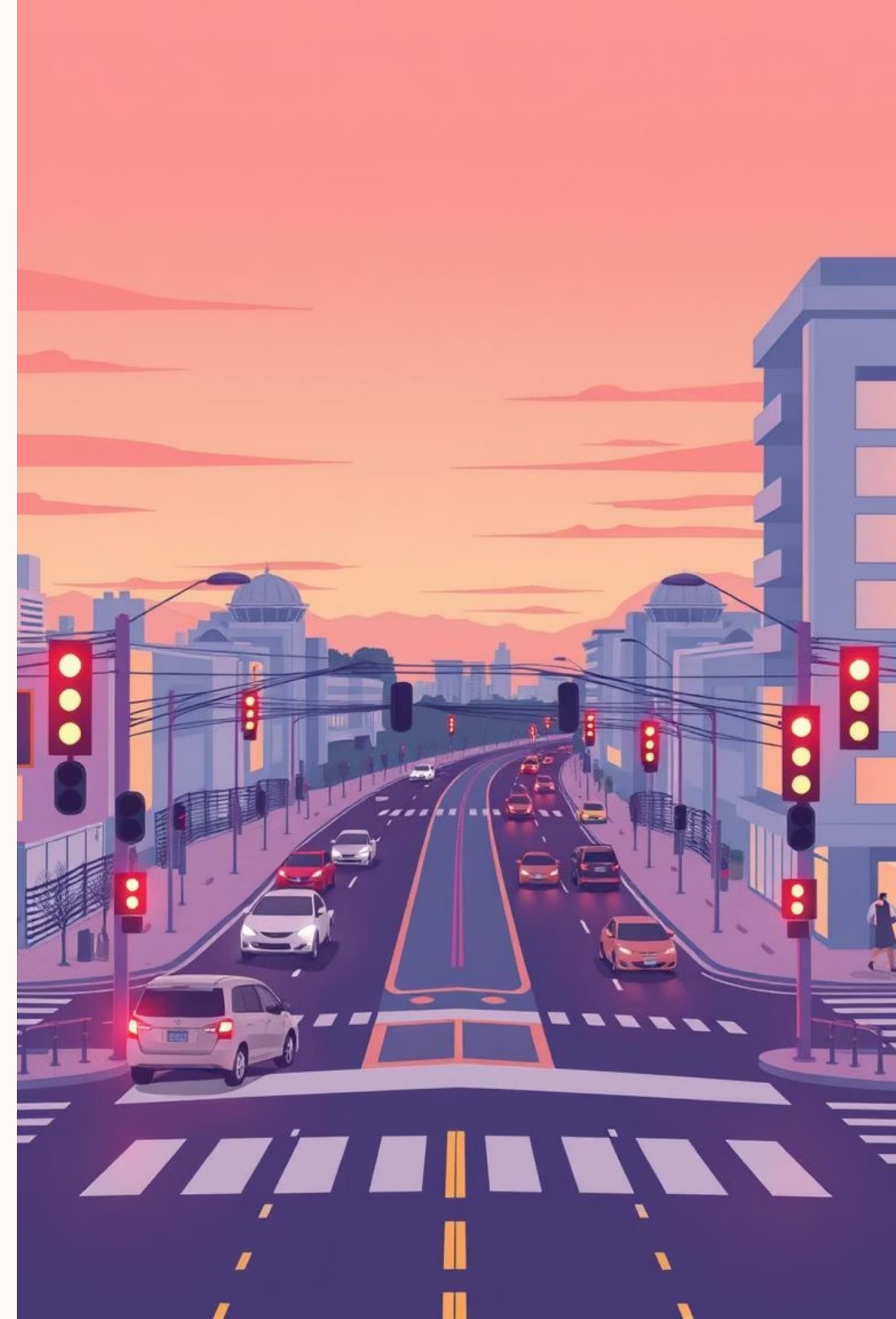


DRLE: Decentralized Reinforcement Learning at the Edge for Traffic Light Control in the IoV

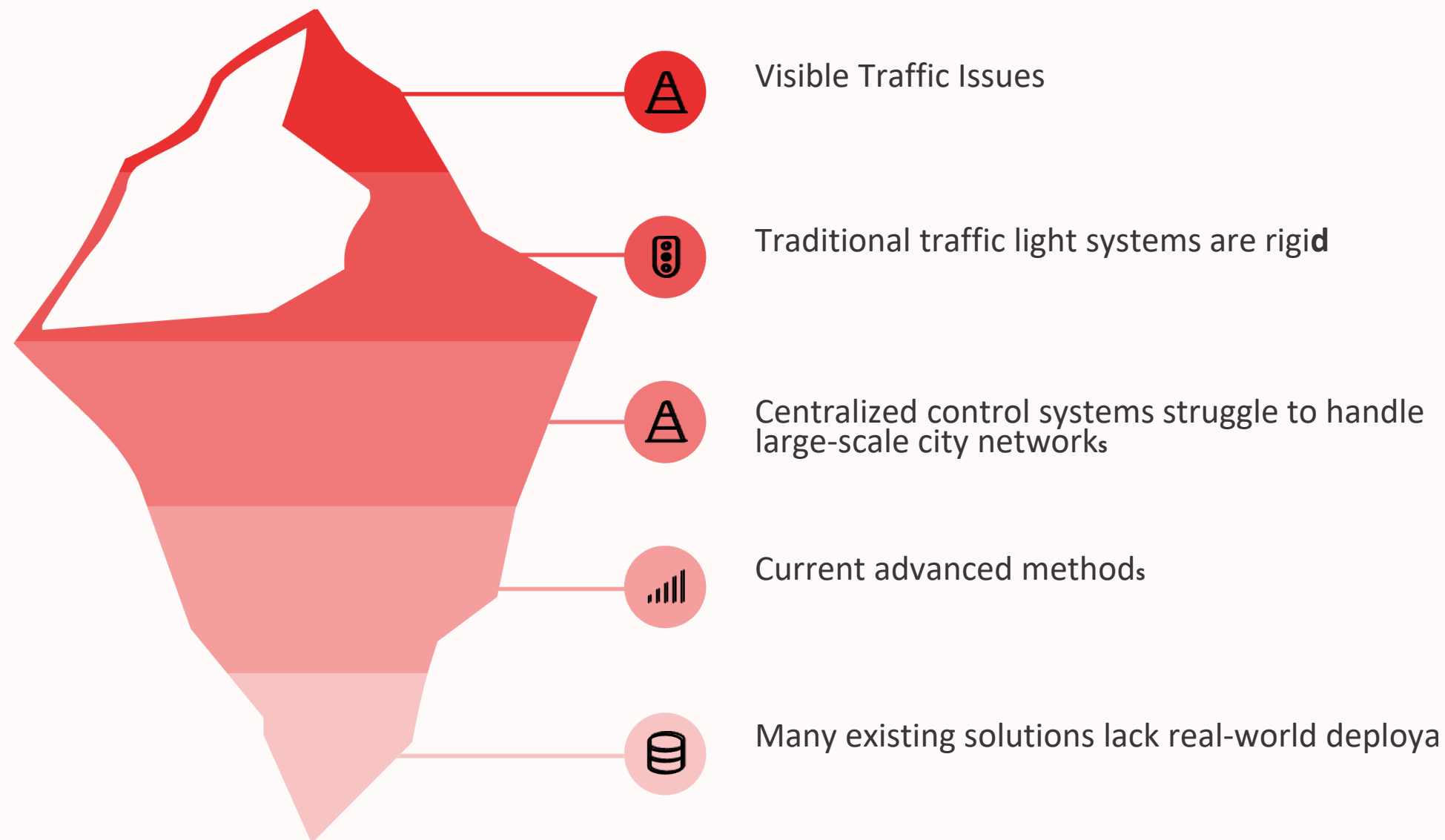
Presented by Pengyuan Zhou, Xianfu Chen, Zhi Liu, Tristan Braud, Pan Hui, Jussi Kangasharju

Team Number : 8



The Urban Traffic Dilemma: A Growing Challenge

Urban traffic congestion is worsening despite advances in connected vehicles and real-time data. Traditional and centralized traffic light systems struggle to adapt quickly or scale efficiently across large networks. Many modern approaches remain limited by high computational demands, reliance on perfect models, or lack of proven real-world performance.



Limitations of Existing Traffic Control Methods

- **Linear Programming** – Optimizes signals using traffic models but needs highly accurate data, lacks learning ability, and can't adapt to unexpected situations.
- **Single-Agent RL** – One controller for all intersections; suffers from huge state/action spaces, making it slow and resource-heavy for large cities.
- **Multi-Agent RL** – Multiple controllers reduce complexity, but existing methods are often not deployable, lack proof of optimality, and use non-standard simulators that limit reproducibility.

Approach Proposed in The Paper

This paper proposes **Decentralized Reinforcement Learning at the Edge (DRLE)** to tackle urban traffic congestion by combining the Internet of Vehicles (IoV) with edge computing. Unlike existing multi-agent RL methods, which often lack real-world deployability, mathematical proof of optimality, and credible large-scale testing, DRLE addresses all three. It uses multiple local agents each controlling its own intersection organized in a three-level edge computing hierarchy. Real-time traffic data from connected vehicles is processed at nearby edge servers, enabling low-latency, scalable, and adaptive traffic light control. The approach is mathematically proven to achieve global optimal performance and is validated through large-scale simulations on open source platforms, showing faster convergence and better efficiency than existing methods.

Why it's an Edge Problem

- **Latency Requirements** – Traffic lights need to respond in real time to changing traffic flow. If all data had to go to a distant cloud server first, delays could make the control decisions outdated. Edge computing processes data closer to intersections, reducing reaction time.
- **Scalability** – A centralized system would have to handle massive amounts of traffic data from an entire city, which becomes slow and inefficient. Edge servers (ES1 and ES2) distribute the load by handling data locally.
- **Local Data Utilization** – IoV devices (vehicles, roadside units) generate valuable data right where the traffic is happening. Processing it at the edge improves accuracy and responsiveness while reducing unnecessary network traffic to the cloud.

State of the Art Literature: Centralized DRL (2018)

Paper: Deep Reinforcement Learning for Traffic Light Control in Vehicular Networks

Authors: Xiaoyuan Liang et al.

His foundational work applied Deep Reinforcement Learning (DRL) with a centralized controller for traffic lights. While effective for single intersections, its limitations became apparent when scaled:

Scalability Issues

Struggled to handle the complexity and data volume of large urban networks.

High Latency

Centralized data transfer introduced significant delays, making real-time response challenging.

Single Point of Failure

A breakdown in the central controller could cripple the entire system

Multi-Agent DRL (2020)

Paper: Multi-Agent Deep Reinforcement Learning for Urban Traffic Light Control in Vehicular Networks

By: Tong Wu et al. Building on centralized DRL, 2020 saw the rise of multi-agent approaches, aiming to distribute control closer to the problem source.

Decentralized Agents

Each traffic intersection was assigned its own independent DRL agent for localized control.

Centralized Training

Despite decentralized execution, agents still relied on a centralized critic during the training phase.

Enhanced Scalability

Significant improvements in handling larger networks and dynamic traffic patterns compared to single-agent systems.

Partial Independence

While more adaptable, the dependency on a centralized critic for training limited true autonomy.

DRLE: Our Contribution to Traffic Control

Paper: DRLE: Decentralized Reinforcement Learning at the Edge for Traffic Light Control in the IoV

DRLE introduces a groundbreaking approach to urban traffic management, leveraging fully decentralized Deep Reinforcement Learning (DRL) directly at the network edge. This eliminates the need for central coordination, fostering resilience and real-time adaptability.

Fully Decentralized

Each traffic light operates autonomously at the edge.

Local Data Reliance

Utilizes real-time data from connected vehicles (IoV).

No Central Authority

Operates without a central controller or critic.

Achieved Performance

Through this innovative architecture, DRLE delivers significant improvements.

Waiting Time Reduction

Achieved in traffic flow optimization upto 52%.

Training Speed

Efficient learning curve.

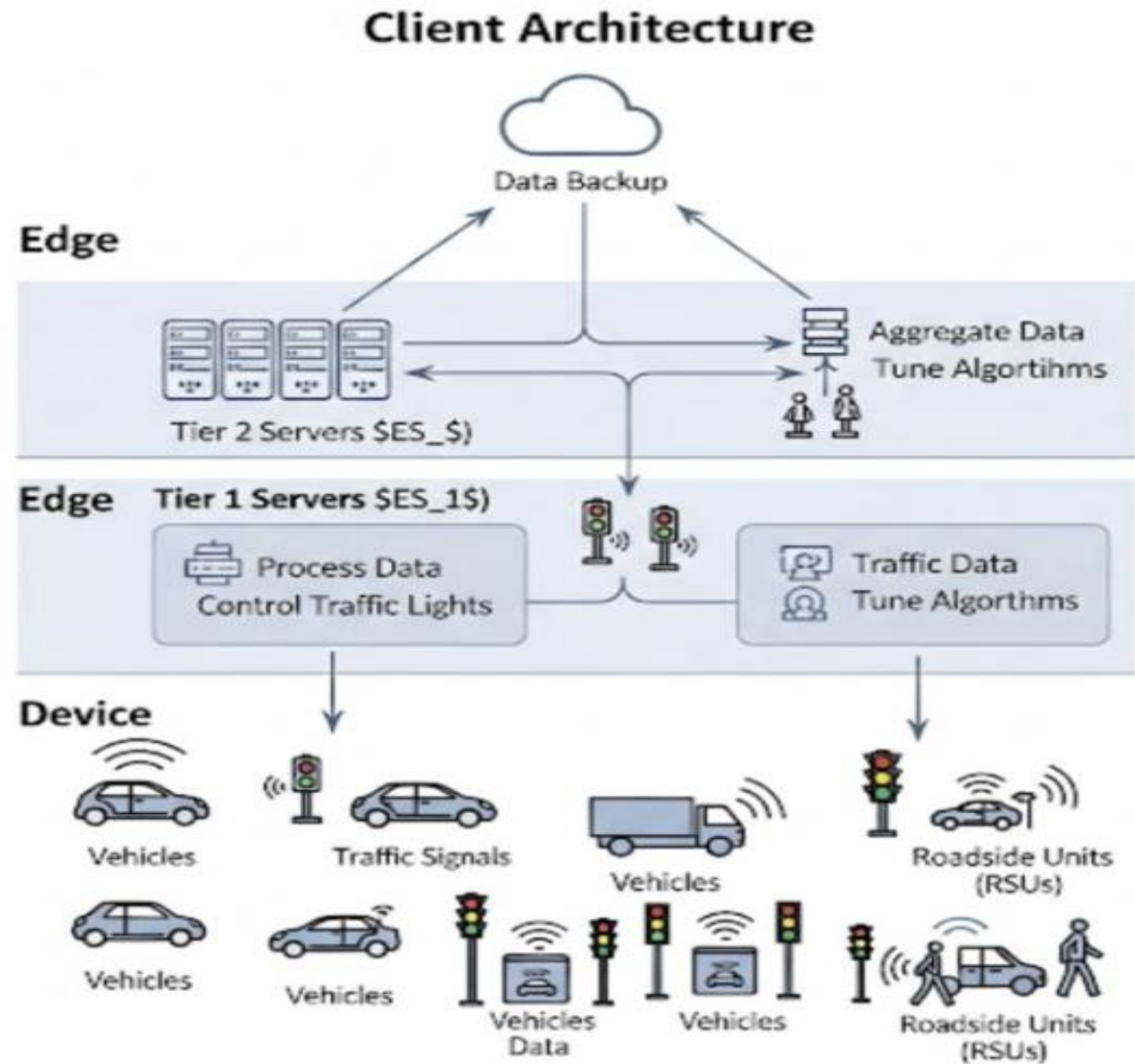
Greenwave Learning

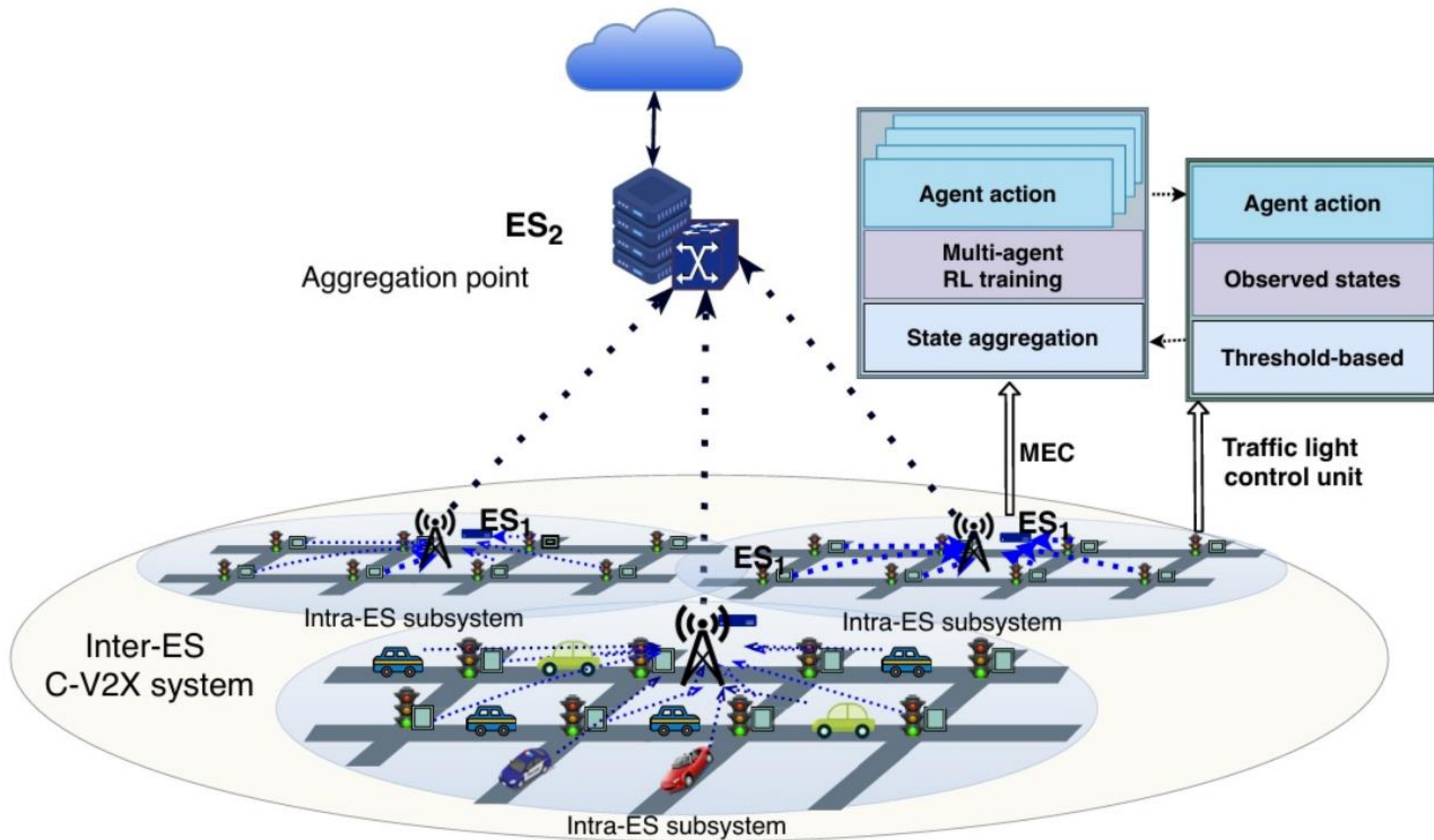
Self-optimizing patterns.

Mathematical Optimality

Guaranteed performance.

Architecture Diagram





Implementation Strategy & Evaluation

- SUMO** → Traffic simulator only. It handles vehicles, intersections, roads, and signal timing.
- Flow** → A Python framework that *connects* SUMO (or other traffic simulators) with reinforcement learning libraries so you can train agents to control traffic lights.
- RLlib** → The reinforcement learning library itself — this is where the DQN algorithm runs.
- NS-3** — Simulates the communication network between vehicles, roadside units, and edge servers, adding delays to make the simulation realistic.



Team Task Allocation for DRLE Development

System Architecture & Edge Deployment (SHIBI KRISHNA RAM)

Design the edge computing architecture:

How RSUs (roadside units) and MEC (Multi-access Edge Computing) servers are placed.

Where DRLE agents run (at the edge vs in central cloud).

Modify the repo so each intersection agent runs locally at RSUs or edge servers instead of only on a central machine.

Set up communication between:

Edge nodes ↔ Central nodes

Intra-ES and Inter-ES

IoV Data Collection & Preprocessing Module (ALANA MARIYA)

Implement vehicle data collection from simulated IoV environment:

Extract vehicle counts, speeds, and queue lengths in real time.

Interface with SUMO or the provided simulator.

Apply data preprocessing at RSUs to reduce bandwidth usage (edge filtering, aggregation, compression).

Edge-based Decision Making & Scheduling (Gautam Krishna M)

- Implement decision-making logic at the edge:
- DRLE model runs locally for each intersection.
- Optimize response time by reducing dependency on cloud computation.
- Add priority scheduling for intersections with emergency vehicles or congestion.
- Compare local decision-making performance vs cloud-based control.

Edge System Optimization and NS3 implementation(Adwaith A)

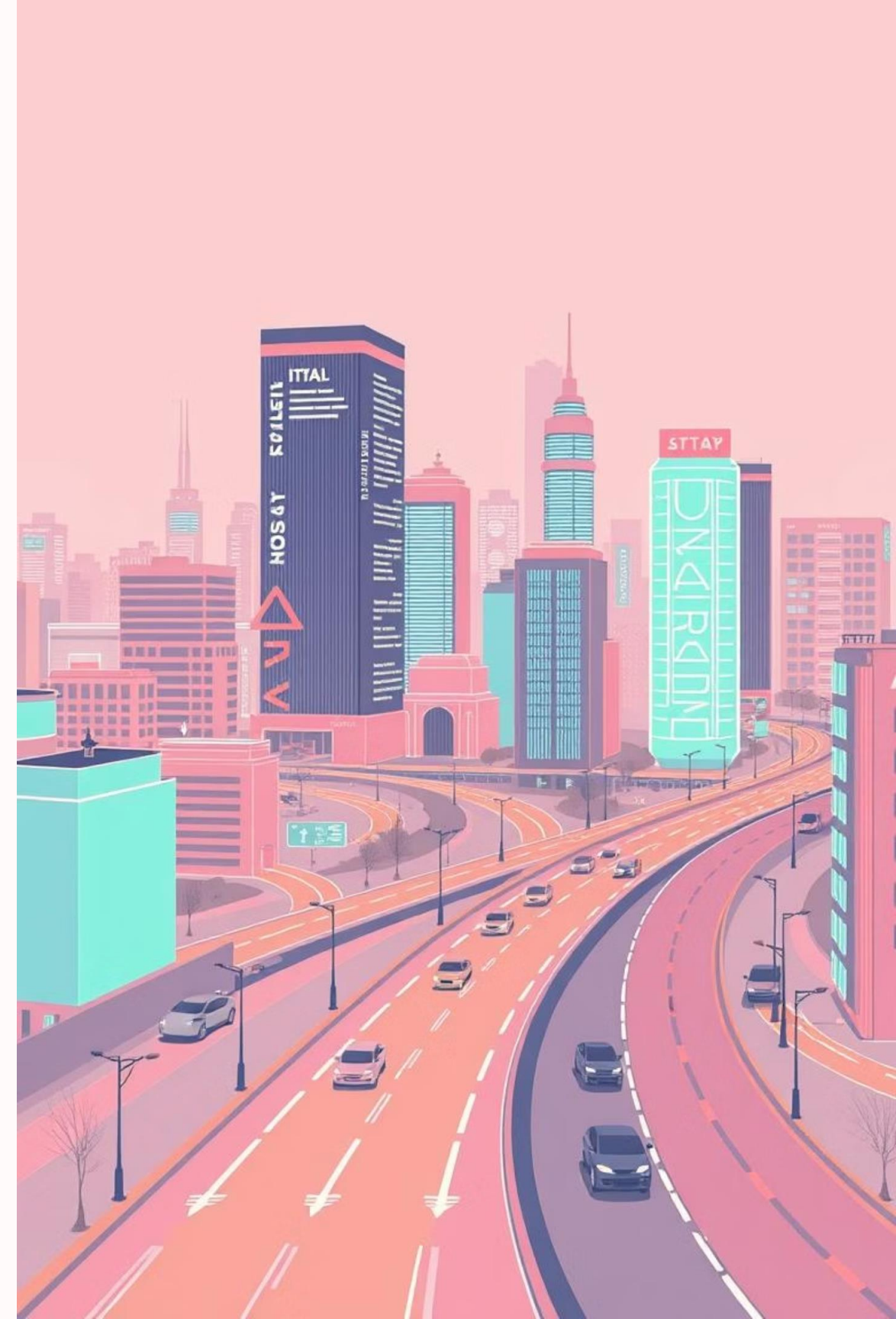
- Develop failover mechanisms if an edge node goes offline.
- Implement backup decision-making at the intersection level.
- Configure ns-3 to simulate IoV communications between vehicles, roadside units, and edge servers.
- Run network topology scenarios matching the experimental setup.
- Collect simulation metrics (latency, packet loss, bandwidth usage) for different configurations.

Q & A

DRLE: The Future of Smart City Traffic

DRLE stands as a scalable, real-time, and intelligent edge-based solution for smart city traffic management, showcasing clear advantages over traditional centralised models through robust theoretical guarantees and comprehensive experimental validation.

This framework marks a significant leap towards truly adaptive and efficient urban mobility systems.



DQN Learning Process in DRLE

Process:

- The DQN agent takes this input and estimates how good each possible action would be
- It uses a neural network to predict these “goodness” values (Q-values) for every possible action.
- The agent then chooses an action sometimes the best-predicted one, sometimes a random one (to explore new strategies).
- After the action is applied, the environment (traffic) changes, and the agent gets feedback in the form of a reward (e.g., reduced queue = positive reward).

Multi-Agent Integration:

- **Intersection Level:** Simple threshold control for immediate response.
- **Intra-ES Level (ES1):** Multiple DQN agents optimize their intersections locally.
- **Inter-ES Level (ES2):** Coordinates ES1 agents and tunes learning parameters in real time.

Why DQN in DRLE?

- Handles large state spaces better than basic Q-learning.
- Learns adaptively from live IoV data.
- Scales well in a decentralized multi-agent setup.

DQN Algorithm Overview in DRLE

What is DQN?

- Deep Q-Network (DQN) is a reinforcement learning method that uses a neural network to estimate Q-values.
- In DRLE, each **intersection** has an agent running DQN to decide the next traffic light phase.

How DQN Works in DRLE

Input:

- Real-time traffic data from IoV devices at the intersection:
- Queue lengths on each approach
- Waiting times of vehicles
- Current traffic light phase
- Number of incoming vehicles